

Adaptive Temporal Action Quantization for Vision-Language-Action Models

Anonymous Author(s)

Affiliation

Address

email

Abstract: Vision-Language-Action (VLA) models have recently shown remarkable progress as generalist robot policies, yet their real-world execution often remains temporally inefficient. A key source of this inefficiency is the temporal structure of teleoperated demonstrations, from which policies often inherit fine-grained execution patterns that require many low-level action steps. Temporal action quantization offers a simple way to accelerate execution by merging low-level actions into temporally coarser action commands, but applying a fixed quantization ratio uniformly across a task introduces a speed-accuracy trade-off. To address this issue, we propose ATQ, a lightweight architectural extension to VLAs that equips a shared action expert with multiple execution modes at different temporal granularities. ATQ uses mode-specific action decoders and a context-conditioned router to select an execution mode at each inference step, while pairing temporal quantization levels with action horizons to control open-loop commitment. Experiments on simulation benchmarks and real-world manipulation tasks show that ATQ improves both task success and execution efficiency, reducing the number of executed action steps while maintaining or improving success rates.

Keywords: Robot manipulation, Vision-Language-Action models

1 Introduction

Vision-Language-Action models (VLAs; [1, 2, 3, 4, 5]) have emerged as a promising framework for generalist robot policies, achieving strong performance by learning from diverse robot demonstrations [6, 7, 8], many of which are collected through human teleoperation. However, teleoperated demonstrations often exhibit conservative, temporally fine-grained behavior: human-in-the-loop control is constrained by limited interface responsiveness and control bandwidth, and human operators adopt a cautious pace during data collection. As a result, VLAs trained on such data often inherit fine-grained execution patterns, taking many low-level action steps even in phases where faster motion is physically feasible.

To mitigate this limitation, recent approaches have explored *temporal action quantization*, which merges a sequence of low-level actions into temporally quantized actions [9]. For example, VLASH [9] shows that temporally quantized actions can approximate the original trajectory with fewer execution steps. While temporal action quantization provides a simple route to faster execution, applying a fixed quantization ratio across an entire task introduces a speed-accuracy trade-off (see Tables 1 and 2): temporally quantized actions are often sufficient during free-space motion, but they can discard trajectory details required for precision-critical phases such as grasping, contact-rich interaction, or release. This suggests that temporal action quantization should be applied selectively according to the current task context.

To this end, we propose ATQ (Adaptive Temporal Quantization), a lightweight architectural extension to VLAs. ATQ equips the shared action expert with mode-specific action decoders and a lightweight

router, which selects an execution mode from the current visual, language, and proprioceptive context at each inference step. Temporally quantized action modes accelerate execution by preserving the net effect of multiple low-level actions with fewer commands, but this approximation can discard intermediate trajectory details and introduce quantization-induced trajectory error. We therefore pair temporal quantization levels with different action horizons, allowing the policy to control how long quantization-induced errors may accumulate under open-loop execution. Concretely, each execution mode is defined by a temporal quantization level and an action horizon, enabling the policy to choose both the temporal granularity of its commands and the commitment length of the resulting action sequence. The entire framework reuses the pretrained VLA backbone and introduces only a small number of additional parameters for the mode-specific action decoders and the router, allowing ATQ to be added to an existing VLA rather than replacing it.

We instantiate ATQ on GR00T N1.5 and evaluate it on simulation benchmarks and real-world manipulation tasks. On simulation benchmarks, ATQ achieves the highest success rate of 68.8% (vs. 65.4%) on RoboCasa-300 over GR00T N1.5 baseline model, while reducing the number of action steps from 321.6 to 274.8. On the real-world PnP Coffee Cup task, ATQ improves the success rate over GR00T N1.5 from 54.2% to 66.7%, while reducing executed action steps from 402.3 to 260.5.

Our contributions are:

- We identify the tendency of VLA policies to inherit fine-grained execution patterns from teleoperated demonstrations as a key source of temporal inefficiency.
- We propose ATQ, a lightweight architecture that models execution modes with different temporal quantization levels and action horizons using mode-specific action decoders and a learned context-conditioned router.
- We demonstrate consistent gains on simulation benchmarks and real-world tasks, achieving a stronger balance between task success and execution efficiency than the base VLA and fixed temporal quantization baselines.

2 Related Work

Vision-Language-Action models (VLAs). VLAs have recently shown great promise in robotic policy learning by leveraging diverse, robot-specific datasets. One line of VLA models [2, 4, 10] discretizes the continuous action space and predicts actions as tokens, showing that pre-trained VLMs [11, 12, 13, 14] can be effectively adapted to low-level action predictions. In contrast, a growing body of studies [3, 5, 15] instead attach action experts that generate continuous action chunks (action chunking) via diffusion or flow-matching, enabling more accurate predictions. For example, GR00T N1.5 [5] conditions a flow-matching head on VLM features to produce action chunks. In this work, we adopt the diffusion modeling formulation for action generation. While most diffusion-based VLAs predict action chunks at a single, fixed temporal granularity inherited from the data collection frequency, this single granularity is not equally suited to all phases of a task. Motivated by this observation, we explore whether jointly learning diverse action trajectories and adapting the action granularity to the current context can lead to faster and more accurate policy learning.

Adaptive Action Granularity. Standard action chunking [16, 17] in diffusion-based VLAs predicts a fixed-length sequence of actions at a single temporal granularity, yet performance is known to be sensitive to this choice, trading off long-horizon foresight against short-term precision. Recent work adapts the granularity rather than fixing it. AAC [18] selects the chunk size at inference time from an entropy-based uncertainty cue, trimming it for uncertain predictions. MoH [19] instead fuses multiple horizons within a single policy through a gating mechanism, and selects stable actions via cross-horizon consensus during inference. From the perspective of execution efficiency, VLASH [9] groups consecutive fine-grained actions into coarser macro-actions at deployment time for faster execution, though its compression rate is fixed per task and chosen manually. In this work, we share the goal of adapting action granularity to context, but differ in how the choice is made and learned.

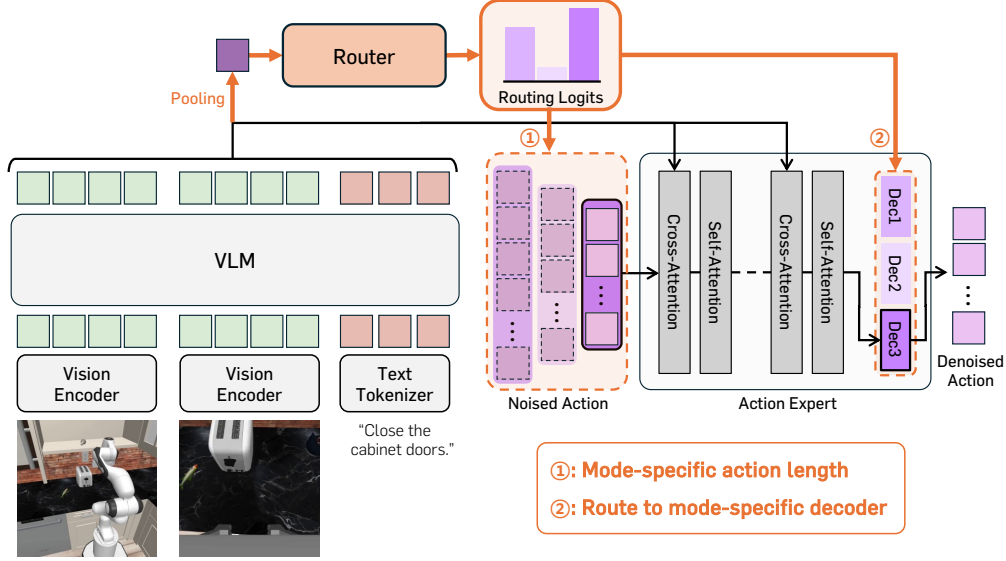


Figure 1: **Overview of the proposed approach.** Overview of ATQ. ATQ augments a shared action expert with mode-specific action decoders and a lightweight router that selects the action-token length and decoder for each execution mode. This enables temporally coarser execution when appropriate while preserving fine-grained control for precision-critical phases.

We jointly train a family of decoders at distinct granularities together with a learned router that selects among them from the current observation and instruction. This lets the model assign coarse predictions to free-space motion and finer ones to demanding, contact-rich phases, while remaining compatible with efficient execution.

3 Method

In this section, we present ATQ (Adaptive Temporal Quantization), a framework that improves rollout speed without sacrificing task performance by equipping a single VLA with multiple action decoders operating at different temporal horizons, together with a learned router that selects among them based on the current context. We first describe the diffusion-based VLA training framework (Section 3.1) and then present ATQ (Section 3.2). An overview of our method is illustrated in Figure 1.

3.1 Vision-Language-Action Models

We build on a pretrained diffusion-based VLA [5]: given a language instruction I , visual observations V_t , and a proprioceptive robot state s_t , a pretrained VLM backbone f_θ produces a context representation $\Phi_t = f_\theta(I, V_t, s_t)$ that conditions an action expert predicting an action chunk $A_t = (a_{t,1}, \dots, a_{t,H})$ of length H . The action expert is a diffusion transformer [20] trained with flow matching [21]: for a clean chunk A_t , a timestep $\tau \in [0, 1]$, and noise $\epsilon \sim \mathcal{N}(\mathbf{0}, I)$, the noised chunk is $A_t^\tau = \tau A_t + (1 - \tau)\epsilon$, and a velocity network $u_\theta(\Phi_t, A_t^\tau, \tau)$ is trained to predict the ground-truth velocity field $A_t - \epsilon$:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{A_t, \tau, \epsilon} \left[\|u_\theta(\Phi_t, A_t^\tau, \tau) - (A_t - \epsilon)\|^2 \right]. \quad (1)$$

3.2 Mixture-of-Decoders Architecture

While standard VLAs commit to a single action horizon with a single temporal granularity, no single granularity fits all phases of a task: contact-rich manipulation demands fine actions, whereas free-space motion can often be handled with coarser ones. To address this problem, we instead train K decoders $\{D_k\}_{k=1}^K$ at different quantization factors q_k under a shared VLM backbone, with a small router selecting which decoder to execute.

111 **Temporal Action Aggregation.** A single action token can summarize several consecutive low-level
 112 actions, advancing the state at a coarser temporal granularity. A decoder with T output tokens
 113 covering $H \geq T$ environment steps is trained on the ground-truth chunk split into windows of size
 114 $q = H/T$, where $q \in \mathbb{N}$ is the temporal quantization ratio, and reduced per window by a function f ,

$$\bar{\mathbf{a}}_i = f(\mathbf{a}_{(i-1)q+1}, \dots, \mathbf{a}_{iq}), \quad i = 1, \dots, T. \quad (2)$$

115 The aggregation function depends on the action dimension: for delta-action dimensions, f sums the
 116 q deltas, since consecutive deltas add up to the net change over the window; for absolute-position
 117 dimensions, f takes the window’s last value $\mathbf{a}_{iq}$, corresponding to the configuration reached at the
 118 end of the window; and for discrete dimensions such as gripper open/close, f also takes the last value.

119 **Decoder set and routing.** We instantiate K action decoders $\{D_k\}_{k=1}^K$, with decoder D_k using T_k
 120 output tokens over a horizon $H_k \geq T_k$, hence quantization ratio $q_k = H_k/T_k$; varying (T_k, H_k) spans
 121 execution modes from uncompressed actions ($q_k = 1$) to temporally quantized actions ($q_k > 1$). A
 122 small MLP router π_ϕ then selects which decoder to run at each chunk, from the average-pooled VLM
 123 context $\bar{\Phi}_t = \text{AvgPool}(\Phi_t)$ and the proprioceptive state $\mathbf{s}_t$. It produces logits $\ell_t = \pi_\phi([\bar{\Phi}_t; \mathbf{s}_t]) \in$
 124 $\mathbb{R}^K$ that a temperature-scaled softmax converts to gating weights:

$$\alpha_t = \text{softmax}(\ell_t/\tau_r), \quad \alpha_t \in \Delta^{K-1}. \quad (3)$$

125 **Training objective.** At each training step we run all K decoders in parallel; this cost is incurred
 126 only at training, since inference runs a single selected decoder. For decoder D_k , we draw a flow-
 127 matching timestep $\tau_k \in [0, 1]$ and noise $\epsilon_k \sim \mathcal{N}(\mathbf{0}, I)$, and form the noised target $\mathbf{A}_t^{(k), \tau_k} =$
 128 $\tau_k \mathbf{A}_t^{(k)} + (1 - \tau_k) \epsilon_k$, where $\mathbf{A}_t^{(k)} \in \mathbb{R}^{T_k \times d_a}$ denotes the ground-truth chunk aggregated for execution
 129 mode k using Equation (2). The decoder predicts a velocity field $\mathbf{u}_\theta^{(k)}$ with loss:

$$\mathcal{L}_{\text{FM}}^{(k)} = \mathbb{E}_{\mathbf{A}_t^{(k)}, \tau_k, \epsilon_k} \left[\frac{1}{T_k d_a} \left\| \mathbf{u}_\theta^{(k)}(\Phi_t, \mathbf{A}_t^{(k), \tau_k}, \tau_k) - (\mathbf{A}_t^{(k)} - \epsilon_k) \right\|^2 \right]. \quad (4)$$

130 The $1/(T_k d_a)$ factor puts per-decoder losses on a common per-element scale, so the shared backbone
 131 and action-expert body—which receive the combined mixture gradient below—are not dominated by
 132 the higher-token decoders.

133 Three terms shape training, with $\mathcal{L}_{\text{FM}, b}^{(k)}$ decoder k ’s loss on batch sample b and $\mathbb{E}_b[\cdot]$ the batch mean.
 134 The *soft-mixture* term weights each decoder’s loss by its routing probability and jointly updates the
 135 backbone, decoders, and router,

$$\mathcal{L}_{\text{mix}} = \mathbb{E}_b \left[\sum_{k=1}^K \alpha_{b,k} \mathcal{L}_{\text{FM}, b}^{(k)} \right], \quad (5)$$

136 where each α_b is floored at ρ and renormalized, $\alpha_b \leftarrow \text{normalize}(\max(\alpha_b, \rho))$, so no decoder is
 137 starved of gradient. The *load-balancing* term pushes the batch-mean routing $\bar{\alpha} = \mathbb{E}_b[\alpha_b]$ toward
 138 uniform, $\mathcal{L}_{\text{bal}} = \|\bar{\alpha} - \frac{1}{K} \mathbf{1}\|_2^2$. The *routing-supervision* term teaches the router to prefer the best-
 139 fitting decoder: from the stop-gradient per-decoder losses, each scaled by an EMA $\bar{\ell}_k^{(\text{ema})} \leftarrow$
 140 $m \bar{\ell}_k^{(\text{ema})} + (1 - m) \mathbb{E}_b[\mathcal{L}_{\text{FM}, b}^{(k)}]$, we form a target q_b and pull the router toward it,

$$q_{b,k} = \text{softmax}_k \left(-\frac{1}{\tau_q} \frac{\mathcal{L}_{\text{FM}, b}^{(k)}}{\bar{\ell}_k^{(\text{ema})}} \right), \quad \mathcal{L}_{\text{sup}} = \mathbb{E}_b [\text{KL}(q_b \parallel \alpha_b)]. \quad (6)$$

141 A lower normalized loss gives a higher target probability; the detached losses make this term update
 142 only the router, and the EMA scaling makes the target reflect each decoder’s *relative* fit rather than its
 143 absolute loss scale. The full objective is $\mathcal{L} = \mathcal{L}_{\text{mix}} + \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} + \lambda_{\text{sup}} \mathcal{L}_{\text{sup}}$. Because $\mathcal{L}_{\text{sup}}$ routes
 144 by fit rather than cost, the speedup is not optimized directly: it emerges because coarse, long-horizon
 145 decoders fit compressible (e.g. action in free-space) segments and are selected there, while fine
 146 decoders are reserved for contact-rich ones. The decoder set $\{D_k\}_{k=1}^K$, target temperature τ_q , and λ_{bal}
 147 together set where the policy lands on the speed/accuracy spectrum; further details are in Section A.1.

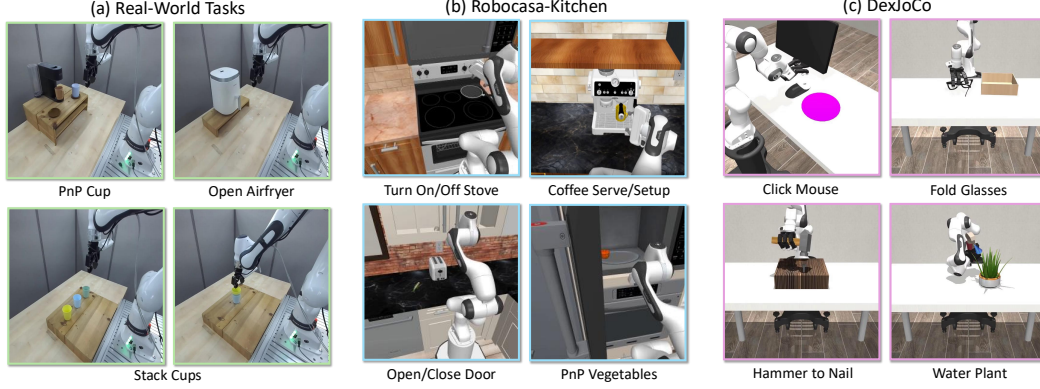


Figure 2: **Experimental Setups.** We evaluate ATQ on (a) real-robot manipulation tasks consisting of pick-and-place, stacking, and articulated object manipulation, utilizing multiple viewpoints, as well as multitask simulation benchmarks of (b) RoboCasa-Kitchen [22] and (c) DexJoCo [23].

148 3.3 Confidence-Gated Inference

149 At test time, two natural rules for selecting the executed decoder are $\arg \max_k \alpha_{t,k}$ (commit to the
 150 most likely decoder) and $k_t \sim \text{Cat}(\alpha_t)$ (sample by router probabilities). Each is sub-optimal on its
 151 own: $\arg \max \arg \max$ tends to select D_1 at most steps and forfeits the speedup compressed decoders
 152 offer, while pure sampling can hurt success rate by picking a coarse decoder in moments that demand
 153 fine control. We instead use a *confidence-gated* rule that interpolates the two,

$$k_t = \begin{cases} \arg \max_k \alpha_{t,k}, & \text{if } \max_k \alpha_{t,k} \geq \kappa, \\ k \sim \text{Cat}(\alpha_t), & \text{otherwise,} \end{cases} \quad (7)$$

154 gated by a confidence threshold κ . When the router strongly prefers one decoder, typically D_1 , we
 155 obey it to preserve task success; when α_t is close to uniform (indicating no clearly preferred mode),
 156 sampling lets the policy exploit the full distribution and gives the decoders for quantized actions a
 157 fair share of selections, recovering their execution-efficiency benefit. Because all decoders are trained
 158 on shared backbone under load balancing, every sampled decoder is in-distribution at every state.

159 4 Experiments

160 We design our experiments to investigate the following questions:

- 161 • Can ATQ maintain performance and accelerate task execution on real-world tasks without compro-
 162 mising success rate? (Table 1)
- 163 • Is ATQ also beneficial even on simulation benchmarks, where the drawbacks of slow trajectories
 164 collected in the real world are less pronounced? (Table 2, Table 3)
- 165 • How does each component of ATQ affect performance? (Table 4)

166 We use the Avg. success rate and Succ-steps (average action steps on successful tasks) as our
 167 evaluation metrics. We consider only successful episodes because failed episodes in simulation
 168 benchmarks often continue rolling out until the timeout limit. Similarly, in real-world evaluations,
 169 some failures also occur due to timeout. This protocol prevents differences in model accuracy from
 170 introducing noise into the measurement of action-step efficiency.

171 4.1 Experimental Setups

172 **Real-World tasks.** We evaluate ATQ on real-world tasks with different conditions. For this, we
 173 tried to prepare both short-horizon and long-horizon tasks with different levels of difficulty (figure 2c).
 174 We designed three static environment tasks: (i) Pick-and-Place Coffee cup, where the robot moves

Table 1: **Real-World evaluation results.** We report the Avg. success rate (% , over 24 trials per task) and Succ-steps (average action-steps on succ-tasks, lower is better) on three Gripper-Real-World tasks: (*Stack Cup*, *PnP Coffee Cup*, *Open AirFryer*), Q denotes the quantization ratio.

Method	Stack Cup		PnP Coffee Cup		Open AirFryer	
	Avg. $\uparrow$	Succ-steps $\downarrow$	Avg. $\uparrow$	Succ-steps $\downarrow$	Avg. $\uparrow$	Succ-steps $\downarrow$
GR00T N1.5	54.2	488.8	54.2	402.3	75.0	156.6
+ fixed quantization Q=2	20.8	239.4	25.0	175.4	58.3	51.3
+ fixed quantization Q=3	29.2	188.7	20.8	210.0	54.2	46.5
+ Ours	62.5	357.5	66.7	260.5	79.2	65.4

Table 2: **RoboCasa-Kitchen benchmark results:** We compare ATQ with baseline methods on RoboCasa-Kitchen. For RoboCasa Kitchen, we report the average success rate (%) and Succ-steps across 24 tasks with models trained using 30, 100, or 300 demonstrations per task. All the results are reproduced by us.

Method	30 demos		100 demos		300 demos	
	Avg. $\uparrow$	Succ-steps $\downarrow$	Avg. $\uparrow$	Succ-steps $\downarrow$	Avg. $\uparrow$	Succ-steps $\downarrow$
π_0 [3]	48.0	302.6	59.3	300.9	62.6	311.2
GR00T N1.5 [5]	47.8	320.3	63.3	321.5	65.4	321.6
+ fixed quantization Q=2	43.8	219.3	57.3	202.6	59.8	214.0
+ fixed quantization Q=3	34.2	233.3	43.4	200.3	49.5	196.5
+ ATQ(Ours)	48.8	272.6	65.5	257.9	68.8	274.8

175 a coffee cup on the machine to chosen side coaster; (ii) Open the airfryer, where the robot simply
 176 opens the machine; and (iii) Stack 3 Cups, where the robot stacks three cups on the middle placed
 177 cup sequentially.

178 **Simulation tasks.** We evaluate ATQ on two different types of simulation benchmarks: (i) RoboCasa-
 179 Kitchen [22], (ii) DexJoCo [23], with different action representation type; (i) Delta-action and (ii)
 180 Absolute-action. To avoid restricting the analysis to a particular data scale, in the case of Robocasa-
 181 Kitchen benchmark, we fine-tuned both the baseline GR00T N1.5 and our method using the 30-demo,
 182 100-demo, and 300-demo subsets of the RoboCasa-Kitchen dataset.

183 4.2 Main Results

184 **Real-World evaluation.** Table 1 reports the results on three real-world manipulation tasks. Com-
 185 pared to GR00T N1.5, ATQ improves the success rate while reducing the action steps for successful
 186 trials on all tasks: from 54.2% to 62.5% (488.8 $\rightarrow$ 357.5 steps) on Stack Cup, 54.2% to 66.7%
 187 (402.3 $\rightarrow$ 260.5 steps) on PnP Coffee Cup, and 75.0% to 79.2% (156.6 $\rightarrow$ 65.4 steps) on Open
 188 the AirFryer. In contrast, fixed-ratio quantization shortens execution but sharply drops the success
 189 rate. With $Q = 2$, the rates fall to 20.8%, 25.0%, and 58.3%, and with $Q = 3$ to 29.2%, 20.8%,
 190 and 54.2% on the three tasks, respectively. This shows that naively compressing every chunk leads
 191 to unreliable execution. These results demonstrate that ATQ provides a more effective trade-off,
 192 accelerating execution while preserving and even improving task success.

193 **Simulation benchmarks.** Table 2 compares ATQ against π_0 , the GR00T N1.5 baseline, and its
 194 fixed-quantization variants. GR00T N1.5 already surpasses π_0 in success rate (e.g., 65.4% vs. 62.6%
 195 at 300 demos), but requires more steps per rollout (321.6 vs. 311.2). Fixed quantization (Q=2, Q=3)
 196 shortens the rollout length but consistently degrades the success rate across all demo settings e.g.,
 197 the 300-demo success rate drops from 65.4% to 59.8% (Q=2) and 49.5% (Q=3). In contrast, ATQ
 198 improves both metrics at once: it achieves the highest success rate across the 30-, 100-, and 300-demo
 199 regimes (48.8%, 65.5% and 68.8% at 30, 100 and 300 demos, vs. 47.8%, 63.3% and 65.4% for
 200 GR00T N1.5 and 48.0%, 59.3% and 62.6% for π_0) while reducing Succ-steps to 272.6, 257.9 and
 201 274.8, well below both baselines.

Table 3: **DexJoCo Single-arm benchmark results.** We compare ATQ with baseline methods on DexJoCo. we report the average success rate (%) and Succ-steps across 6 tasks with models trained using single-arm dataset. All the results are reproduced by us.

Method	DexJoCo	
	Avg. $\uparrow$	Succ-steps $\downarrow$
π_0	73.7	396.2
GR00T N1.5	73.0	403.5
+ fixed quantization Q=2	55.3	330.2
+ fixed quantization Q=3	40.0	332.1
+ ATQ (Ours)	75.3	374.6

Table 4: **Ablation study.** Average success rate (%) and action-steps required on RoboCasa (300 demos) when selectively enabling different components of ATQ. Our default setting is highlighted in gray .

Method	Avg. $\uparrow$	Succ-steps $\downarrow$
Argmax	70.2	312.8
Sampling	66.8	273.0
Random	65.3	267.8
Conf-gated	68.8	274.8

(a) Router decision.

Method	Avg. $\uparrow$	Succ-steps $\downarrow$
Balance + KL	68.8	274.8
w/o KL	65.7	265.6
w/o Balance	66.7	265.3
w/o both	64.8	273.6

(b) Loss design.

Method	Avg. $\uparrow$	Succ-steps $\downarrow$
Attention pool	64.3	262.9
Last token	65.2	271.7
Avg pool	68.8	274.8

(c) Router input.

Action horizon	Quant. ratio	Avg. $\uparrow$	Succ-steps $\downarrow$
(16, 16, 8)	(1, 2, 2)	63.7	257.4
(16, 16, 16)	(1, 2, 4)	60.5	249.1
(16, 16, 16, 16)	(1, 2, 4, 8)	57.4	249.8
(16, 16, 8, 8)	(1, 2, 1, 2)	68.8	274.8

(d) Decoder set.

Table 3 shows ATQ also improves both metrics at once on the DexJoCo single-arm benchmark, where ATQ is compared against π_0 , GR00T N1.5, and its fixed-quantization variants. Fixed quantization reduces the number of steps but sharply lowers the success rate (73.0% $\rightarrow$ 55.3% for Q=2 and 40.0% for Q=3). ATQ, on the other hand, achieves the highest success rate of 75.3%, surpassing both GR00T N1.5 (73.0%) and π_0 (73.7%), while requiring fewer steps than either (374.6 vs. 403.5 and 396.2), again delivering both higher accuracy and faster execution.

4.3 Analysis and Ablation Study

Qualitative Analysis: Does the Router’s Choice Make Sense? The training objective constrains the router through aggregate quality and load-balance signals; it doesn’t tell the router explicitly when a coarser decoder for quantized action is safe. As a small sanity check, we record the router’s per-step pick on successful simulation rollouts and visualize the summed probability of the coarser decoders over time, alongside a few frames sampled across the episode and the frame at which this probability is the lowest (Figure 3).

In the episodes we visualize, this probability tends to stay in its upper range during free-space transit and to drop as the gripper approaches the object for grasp or release, with the lowest-probability frame landing near the grasp or release moment. In other words, in regions where action quantization is plausibly unsafe, the router lowers its preference for the coarser decoders, which is one piece of qualitative evidence that the routing policy learned by our method behaves in line with its intended role.

Ablation study. We ablate the key components of ATQ on RoboCasa (300 demos); results are summarized in Table 4. For *router decision* (Table 4a), argmax attains the highest success rate but yields little speedup, while our confidence-gated rule recovers meaningful acceleration with only a

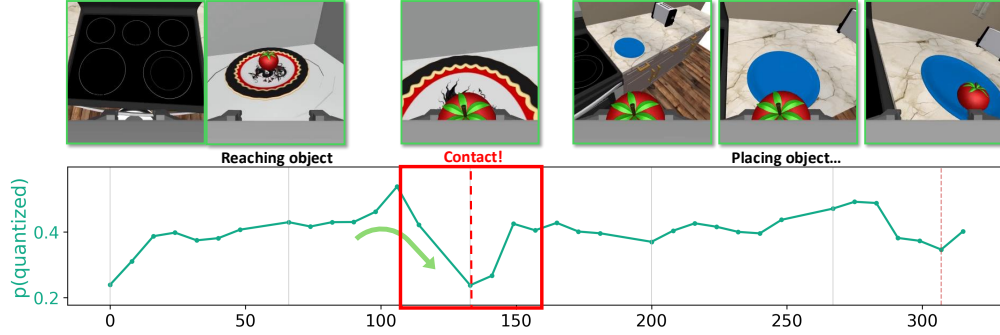


Figure 3: **Router pick analysis.** We show the probability of selecting the action decoder that produces quantized actions over time, shown alongside the corresponding scenes from Robocasa-Kitchen simulation tasks.

slight drop, still outperforming the baseline. For *loss design* (Table 4b), removing the load-balancing or KL routing term degrades performance, confirming both are necessary. For *router input* (Table 4c), the choice of VLM-feature pooling has marginal effect on Succ-steps, so we adopt the variant with the best success rate (average pooling). For *decoder set* (Table 4d), simply adding a higher-quantization decoder hurts performance, whereas diversifying the action horizon improves it by mitigating the accumulation of quantization-induced trajectory errors.

Inference Cost. Table 5 compares the inference latency, averaged over 100 runs on an RTX 5090 GPU. ATQ adds a marginal overhead of 1.83 ms over the baseline ($\times 1.05$), confirming that our architecture does not substantially increase inference cost.

Table 5: **Inference cost.** Inference Latency on RoboCasa.

Method	Latency (ms) ↓
GR00T-N1.5	37.73
Ours	39.56 ($\times 1.05$)

5 Conclusion

In this work, we proposed ATQ, an adaptive temporal action quantization architecture that augments a pretrained VLA with a mixture of decoders operating at diverse quantization levels and action horizons, together with a learned router that selects an execution mode from the current context. Rather than applying action quantization indiscriminately, ATQ assigns coarse, long-horizon decoders to compressible free-space motion and reserves fine-grained decoders for contact-rich phases, deciding when coarse execution is safe. Across both simulation benchmarks (RoboCasa-Kitchen, DexJoco) and real-world manipulation tasks, ATQ consistently improves task success rate while substantially reducing the number of action steps required for completion, adding only marginal inference overhead. These results demonstrate that bringing adaptive action quantization into policy learning improves both task success and execution speed over the base VLA, offering a lightweight and practical path toward faster, more reliable VLA execution.

6 Limitation

Our current method infers the feasibility of action quantization from VLM features and realizes this behavior through router training. However, this remains an implicit signal rather than a ground-truth indication of whether an action chunk can truly be quantized. Future research could construct such ground-truth supervision by deliberately applying action quantization during real-robot task execution and labeling observations as quantization-compatible according to the resulting task success or failure. Moreover, if action quantization becomes a more widely adopted execution paradigm, it may motivate robot manipulation system-VLA architecture co-design, where the robotic system and policy architecture are jointly designed to provide explicit signals about the permissible degree of action quantization.

References

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025.
- [4] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [5] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [6] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. W. He, V. Myers, M. J. Kim, M. Du, et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pages 1723–1736. PMLR, 2023.
- [7] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *IEEE International Conference on Robotics and Automation*, pages 6892–6903. IEEE, 2024.
- [8] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [9] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han. Vlash: Real-time vlas via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.
- [10] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Thompson, Q. Vuong, T. Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [11] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [12] G. Chen, Z. Li, S. Wang, J. Jiang, Y. Liu, L. Lu, D.-A. Huang, W. Byeon, M. Le, M. Ehrlich, et al. Eagle 2.5: Boosting long-context post-training for frontier vision-language models. *Advances in Neural Information Processing Systems*, 38:91077–91100, 2026.
- [13] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [14] X. Chen, J. Djolonga, P. Padlewski, B. Mustafa, S. Changpinyo, J. Wu, C. R. Ruiz, S. Goodman, X. Wang, Y. Tay, et al. Pali-x: On scaling up a multilingual vision and language model. *arXiv preprint arXiv:2305.18565*, 2023.

- 300 [15] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, et al.
301 Cogact: A foundational vision-language-action model for synergizing cognition and action in
302 robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.
- 303 [16] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation
304 with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- 305 [17] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion
306 policy: Visuomotor policy learning via action diffusion. the intl. *Journal of Robotics Research*,
307 2023.
- 308 [18] Y. Liang, X. Wang, K. Wang, S. Wang, X. Peng, H. Chen, D. K. H. Chua, and P. Vadakkepat.
309 Adaptive action chunking at inference-time for vision-language-action models. *arXiv preprint*
310 *arXiv:2604.04161*, 2026.
- 311 [19] D. Jing, G. Wang, J. Liu, W. Tang, Z. Sun, Y. Yao, Z. Wei, Y. Liu, Z. Lu, and M. Ding. Mixture
312 of horizons in action chunking. *arXiv preprint arXiv:2511.19433*, 2025.
- 313 [20] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the*
314 *IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
- 315 [21] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative
316 modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- 317 [22] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu.
318 Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science*
319 *and Systems*, 2024.
- 320 [23] H. Wang, W. Zhao, X. Wang, S. Huang, H. Lin, B. Zheng, R. Xu, G. Wang, Y. Mu, H. Wang,
321 et al. Dexjoco: A benchmark and toolkit for task-oriented dexterous manipulation on mujoco.
322 *arXiv preprint arXiv:2605.16257*, 2026.

323 A Experimental Details

324 A.1 Implementation Details

325 We report the performance of ATQ by fine-tuning the pretrained Vision-Language-Action model
326 GR00T N1.5, following its official implementation. We freeze the VLM backbone and only train the
327 action expert. All models are trained jointly on a consolidated dataset combining the training data
328 from all tasks, with a batch size of 64 for 60k iterations. We set the coefficient of the balance loss
329 to $\lambda_{\text{bal}} = 0.05$ and KL loss to $\lambda_{\text{sup}} = 0.1$. We set the threshold to $\kappa=0.7$. The remaining training
hyperparameters are summarized in Table 6.

Table 6: Hyperparameter details for training ATQ.

GR00T N1.5 + ATQ	
Optimizer	AdamW
Optimizer momentum	$\beta_1, \beta_2 = 0.95, 0.999$
Optimizer weight decay	$1\text{e-}5$
Learning rate	$1\text{e-}4$
Learning rate scheduler	Cosine decay
Warmup iterations	3,000
Batch size	64
Training iterations	60,000

330